

Project - Algorithm Design

Learning Objectives

- Design a complete algorithmic solution for a real-world problem
- Apply all algorithmic constructs (sequence, selection, iteration) in a project
- Document and present algorithmic solutions with flowcharts and pseudocode

Engage (5-7 minutes)

Present the project brief: design an algorithm for a smart cafeteria system that handles student registration, daily menu display, order placement, payment processing, and order tracking. Discuss how this requires combining all algorithmic concepts learned—sequential steps, conditional logic, loops for menu management, and searching for order lookup.

Explore (10-12 minutes)

In teams of 3-4, students select a project domain (cafeteria, library, parking, or events). They decompose the problem into sub-problems, design flowcharts for each module, write pseudocode, analyze complexity, and identify the most critical algorithmic challenge. Teams maintain a project journal documenting design decisions and trade-offs.

Explain (10-12 minutes)

This project synthesizes Chapter 5 concepts: problem decomposition breaks the system into modules; sequential algorithms define the main workflow; selection handles user choices and business rules; iteration manages repeated operations (menu loops, order lists); sorting and searching organize data; efficiency analysis justifies algorithm choices. The design document must include: problem statement, decomposed modules, flowcharts, pseudocode, complexity analysis, and testing strategy.

Elaborate (8-10 minutes)

Teams refine their solutions through iterative design: first draft, peer feedback session, revision, and final presentation. They consider edge cases (system crashes during payment, invalid orders, simultaneous access). Documentation standards are introduced—how professional developers write algorithmic specifications that other team members can implement.

Evaluate (5-8 minutes)

Final assessment: teams present their complete algorithmic design to the class. Rubric covers: problem decomposition quality (20%), algorithm correctness and completeness (30%), use of appropriate constructs (20%), complexity analysis (15%), and presentation clarity (15%). Individual reflection: each student writes what they would improve about their team's design.